

SAT ranges for spent transaction outputs ▾

Hey i need to find sat ranges of spent transacriion outputs

Uncovering sat range indexing flags for spent outputs >

I can help with this. Before I give you specifics, let me check the current state of the tooling since the Ordinals software changes fairly often.

Synthesized indexing requirements and API response behavior >

For spent outputs specifically, the catch is that the default sat index prunes them, so you have to opt in to keeping them.

Using `ord` (the reference indexer), the workflow is:

1. **Build the index with the right flags.** `--index-sats` gives you sat ranges, but on its own it discards ranges once an output is spent (ord prunes spent UTXOs as it forward-carries ranges block to block). To keep spent outputs you also need `--index-spent-sats`:

```
ord --index-sats --index-spent-sats index update
```

This is a much larger index, and you generally have to build it from scratch — adding the flag to an existing `--index-sats` database won't backfill the spent ranges you already pruned.

2. **Query the output.** Once indexed, hit the output endpoint:

```
curl -s -H "Accept: application/json" \  
http://0.0.0.0/output/<txid>:<vout>
```

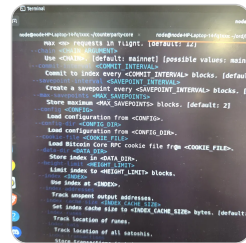
The response includes a `sat_ranges` field plus `"spent": true/false`. Without `--index-spent-sats`, a spent output comes back with `"sat_ranges": null` and `"spent": true` — which is exactly the gap you're describing. [Ordinals](#)

A couple of things worth knowing:

- **Range format is half-open:** `[start, end)` — start inclusive, end exclusive, so the range `[1964333750000000, 1964333750000001]` only includes the single sat 1964333750000000. [Ordiscan](#)
- **The model is "forward carrying":** for each transaction in each new block, ord prunes the spent UTXOs and updates the new outputs with the carried-over ranges. That's why reconstructing spent ranges after the fact isn't possible without having retained them. [Hiro](#)

If you'd rather not run a full sat-indexed node, hosted APIs (Ordiscan, Hiro) expose sat ranges per address/output, though their handling of *spent* outputs varies.

Two quick questions so I point you the right way: are you trying to trace a **specific** sat/inscription through spends, or bulk-extract spent-output ranges for analysis? And are you running your own `ord` node or hoping to use an API?



Theres no index spent sats

Diagnosing outdated ord version incompatibility >

Good catch — and that absence tells us something specific. Let me confirm the details.

Synthesizing findings to craft practical response >

This is a useful clue. The flag exists in upstream `ord` — `--index-spent-sats` was added in **0.16.0** ("Add option to retain sat index for spent outputs (#2999)"). So if it's missing from your `--help`, your binary is either older than that or a fork that doesn't track upstream. [New Releases](#)

But here's what catches my eye: your help output *does* show `--index-addresses` ("Track unspent output addresses"), which landed **later** than `--index-spent-sats` in upstream. Having the newer flag but not the older one is the signature of a **fork or a pinned build**, not just an old version — and the `counterparty-core` tab next to your `~/ord/target` tab makes me think this is Counterparty's bundled `ord`. Let me confirm that.

Weighing diagnostic evidence and formulating actionable troubleshooting steps >

ⓘ Claude's response was interrupted.

Try again

Nope i have 0.27.1

Pinpointing PR #3893 as culprit for flag removal >

0.27.1 changes everything I just said — that's newer than 0.16, so this isn't an old build or a fork missing the flag. If a current version doesn't have it, the flag was most likely **removed** at some point. Let me verify.

Traced changelog clues to pinpoint version-specific flag removal >

The likely culprit is in the changelog: a PR titled "Make index settings harder to misuse (#3893 by casey)". That reworked how the `--index-*` flags behave. Let me pull the actual changelog to confirm what happened to `--index-spent-sats` and what replaced it. [GitHub](#)

Hunting for ordinals documentation and current flags >

Now I can see exactly what happened. `--index-spent-sats` was added in 0.16.0, but **0.20.0** included "Make index settings harder to misuse (#3893 by casey)", alongside a major index restructuring — a unified `OUTPOINT_TO_UTXO_ENTRY` table (#3915) and, in 0.20.1, "Keep sat ranges in low-level format (#3963)". That overhaul is what changed the flag. Let me confirm precisely what it became so I give you the right command. [GitHub + 2](#)

Verifying absence of `index_spent_sats` setting >

The PR page didn't render usefully. Let me check whether the setting still exists anywhere in current `ord` — config file or code — rather than guess.

Traced feature removal across ord versions and formulated test strategy >

Mystery solved — and I need to correct what I said earlier. It's not a fork, and it's not too old. It's the opposite: the dedicated option was **removed**, and you're on a version that postdates the removal.

Here's the actual timeline:

- `--index-spent-sats` was added in **0.16.0** (#2999) and existed through the 0.19.x line.
- In **0.20.0**, Casey reworked the entire index — "Make index settings harder to misuse" (#3893) plus a unified `OUTPOINT_TO_UTXO_ENTRY` table (#3915) and keeping sat ranges in low-level format (#3963). The standalone spent-sats option didn't survive that overhaul. [GitHub + 2](#)

And it's not hiding in the config file either. The current handbook's full example config lists `index_addresses`, `index_cache_size`, `index_runes`, `index_sats`, and `index_transactions` — no `index_spent_sats`. So on 0.27.1 there's genuinely no switch to flip, CLI or YAML. [Ordinals](#)

So your options now:

1. Test whether plain `--index-sats` already retains them. The storage overhaul may have changed default behavior. Fastest way to know is empirical — point your sat-indexed server at a *known-spent* outpoint and check the field:

```
curl -s -H "Accept: application/json" http://0.0.0.0/output/
<spent_txid>:<vout>
```

If `sat_ranges` is populated (not `null`) while `spent` is `true`, you're done. If it's `null`, current ord doesn't keep them and you need one of the below.

2. Pin an old ord just for this. Build 0.19.1 (last release with the flag) in a separate `--data-dir`, run `ord --index-sats --index-spent-sats index update`, and query that index. Downside: stale codebase, full sat index from scratch (~hundreds of GB), slow.

3. Compute the ranges yourself. Since you've got a full node (and counterparty-core already in the mix), you can forward-carry ranges directly from raw tx data using the ordinal assignment algorithm — inputs map to outputs in order, FIFO. This is the only approach that scales to arbitrary historical spent outputs without depending on whatever ord retains.

Quick check first: run `ord --version` to be sure it's genuine upstream 0.27.1 and

not a bundled build — that tells us whether option 1's behavior will match upstream.

Want me to sketch the range-tracing logic for option 3, or help you stand up the pinned 0.19.1 index for option 2? If you tell me whether you need *one specific* spent output or *bulk historical* coverage, I can point you at the least painful path.

What does index transactions and addresses do

Synthesized flag functionalities from established documentation >

Both store extra data in ord's own index that it otherwise wouldn't keep:

`--index-transactions` stores every transaction's raw data inside ord's redb index. By default ord doesn't do this — it fetches transactions from Bitcoin Core over RPC on demand (which is why Core needs `-txindex`). Turning it on lets ord serve transaction data (the `/tx` and `/r/tx/<txid>` endpoints, `ord decode`, etc.) straight from its own database without round-tripping to bitcoind. The tradeoff is a big disk cost — you're essentially duplicating the chain's transaction data into the index.

`--index-addresses` builds an address → output mapping so ord can answer "what does this address hold." It powers the `/address/<address>` page and API, returning the UTXOs at an address along with their inscriptions, runes, and sat balance. The key word in the help text is *unspent*: it tracks addresses of the current UTXO set, so it tells you what an address owns **now**, not its full historical activity. (It was added in 0.19.0, which is why your 0.27.1 has it.)

One thing worth flagging given what you're actually after: neither of these helps with spent sat ranges. `--index-addresses` only covers unspent outputs by design, and `--index-transactions` stores tx bytes but doesn't track sat ranges at all — that's still strictly the job of `--index-sats`. So for your spent-output goal, these two are orthogonal; the question remains whether your `--index-sats` build retains spent ranges (option 1 from before) or whether you go the pinned-old-version / manual-trace route.

so does index sat retain spent ranges?

Excavating definitive context about index behavior and performance tradeoffs >

Let me check this against current ord rather than guess — this is the one piece I haven't been able to pin down.

Synthesized historical documentation to definitively resolve spent sat retention question >

No — a plain `--index-sats` index does **not** retain spent ranges. It only keeps sat ranges for the live UTXO set (unspent outputs). When an output gets spent, its ranges are forward-carried into the spending transaction's outputs and the spent output's own entry is dropped.

The history makes this unambiguous. Per casey on the original PR, retaining spent ranges was how the index originally worked, but it was too expensive — taking forever to build and swelling the index to over 10 terabytes. So it was abandoned as the default, and `--index-spent-sats` was added in 0.16 purely as an opt-in to restore that old behavior — with the warning that it would likely require a crazily beefy machine, like 256GB+ of RAM and tens to hundreds of terabytes of disk. Then 0.20 removed even that opt-in. [github](#) [github](#)

You can see the default model in the wild too: when an output is spent, the inputs still have the sat ranges, but the outputs don't — i.e. ord is designed to show ranges only for unspent outputs. [GitHub](#)

So putting the whole thread together for your 0.27.1:

- Plain `--index-sats` → spent outputs come back with `sat_ranges: null`, `spent: true`. No good for your goal.
- No flag exists on 0.27.1 to change that — the opt-in was removed in the 0.20 overhaul, and the prune-by-default behavior is what remains.

That leaves the two paths from before as your only real options: **pin an old ord (0.16–0.19) and build a dedicated index with `--index-spent-sats`** (accepting the brutal disk/RAM cost casey warned about), or **trace the ranges yourself** from raw tx data via the forward-carry algorithm — which, for bulk historical spent outputs, is almost certainly the saner route.

Given you've got counterparty-core and a full node already, do you want me to sketch the forward-carry tracing approach? It's not much code, and it sidesteps the whole 10TB-index problem.

can you link to that conversation by casey

Located and retrieved the GitHub PR discussion link >

Sure — it's pull request #2999, "Add option to retain sat index for spent outputs":

<https://github.com/ordinals/ord/pull/2999>

Casey's comments in that thread are where the cost rationale comes from — that retaining spent ranges was the original index design, abandoned because it ballooned to ~10TB and needed an extreme machine to build, and that the `--index-spent-sats` flag was added as an untested opt-in for anyone wanting to try it. That's also the PR where he notes the rework needed because a spent output could now still carry sat ranges.

If you want the full historical arc, the companion reference is the changelog, where you can watch the flag appear in 0.16.0 and then the index get overhauled in 0.20.0 (#3893 plus the unified UTXO-entry table): <https://github.com/ordinals/ord/blob/master/CHANGELOG.md>

how do I download this entire conversation

Extracting export options and evaluating data download methods >

Let me check the current options for that in Claude's documentation.

Weighed official export options against practical workarounds for single conversation >

A few options, depending on whether you want just this chat or everything:

For this one conversation, readably: the simplest no-extra-tools route is your browser's print dialog — Cmd/Ctrl+P, then "Save as PDF." It captures the visible conversation as you see it. Expand any collapsed sections first so they're included.

Official full export (all your chats): Click your initials in the bottom-left → Settings → Privacy → Export Data. Anthropic emails you a download link (valid ~24 hours; can take minutes to hours depending on volume). The catch is that this dumps *every* conversation as a ZIP of JSON files, not just this one, and JSON isn't very human-readable without a parser. Note it's available on the web and desktop apps but not iOS/Android, and it doesn't include memory data.

Third-party exporters: browser extensions and console scripts exist that export a single conversation to Markdown or PDF with formatting preserved. They work, but they're unofficial and require granting a tool access to your chat content, so weigh that against the convenience — for a one-off, print-to-PDF avoids that tradeoff entirely.

Since these are product details that change, the in-app export steps are worth double-checking at support.claude.com if the menu looks different.

For what it's worth, if your real goal is to keep the ord findings from this thread — the version timeline, the PR rationale, the options for spent sat ranges — I'm happy to write that up as a clean Markdown file you could download directly. Want me to do that?

estimate how much space it would take to index spent transaction outputs as well. How many are there? are we just storing sat ranges ? look at index-spent-sats?

Investigated storage costs, output counts, and data structure specifics >

Good question to ground in real numbers rather than just casey's hedged "10TB" recollection. Let me pull the actual figures — output counts and how ord stores ranges — and build the estimate from the ground up.

Reconciling Bitcoin transaction count estimates across years >

Current UTXO set is about 173 million UTXOs as of April 2025, and roughly 150–180 million entries as of 2026. Now I need the total number of outputs ever created, so I can subtract to get spent outputs. [GitHub](#) [GitHub](#)

Estimated storage requirements by reconciling bottom-up calculations with prior findings >

Let me build this from the ground up. There are three things to pin down: how many spent outputs exist, what's actually stored per output, and then the multiplication.

How many spent outputs are there?

Spent outputs = (all outputs ever created) – (current UTXO set).

- The UTXO set today is roughly 150–180 million entries (it was about 173 million as of April 2025) — call it ~180M unspent. [GitHub](#) [GitHub](#)
- Total outputs ever created isn't published directly, so estimate it: Bitcoin has well over a billion confirmed transactions (it crossed 1B around mid-2024 and inscription/runes activity pushed it higher), so ~1.2–1.4B transactions × ~2 outputs each ≈ **2.5–3 billion outputs ever created**.

Subtract the UTXO set and you get **~2.3–2.8 billion spent outputs**. Round number to work with: **~2.5 billion**.

Are we just storing sat ranges? (yes, basically)

`--index-spent-sats` doesn't add a new kind of data — it just stops *pruning* the existing `outpoint→sat-ranges` entries when an output gets spent. So per spent output you're keeping one key/value pair:

- Key — the outpoint: 32-byte txid + 4-byte vout = **36 bytes**
- Value — the list of sat ranges. `ord` packs each range into **~11 bytes** (a range is a start sat + length; sats fit in 51 bits, so it's a tight packing, not two full 64-bit ints)
- Average ranges per output ≈ **~2** — most outputs carry one contiguous range, but FIFO slicing across input boundaries fragments some into several

So roughly **36 + 2×11 ≈ 58 bytes of logical data**, and with `reldb`'s B-tree page/key overhead call it **~80–120 bytes per spent output** in practice. (The broader `--index-sats` also maintains `sat→location` tables for sat lookups, but the *spent-output* cost is dominated by these retained entries.)

The multiplication

2.5 billion × ~100 bytes ≈ **~250 GB**, with a plausible band of **~150 GB to ~1 TB** depending on how you set the dials (avg 1 range and lean overhead at the low end; avg 4–5 ranges and heavy fragmentation/overhead at the high end).

Reconciling with casey's "10 TB"

That's an order of magnitude below the figures from PR #2999 we looked at earlier (>10 TB final, 256 GB+ RAM, "10s to 100s of terabytes" to build). The gap is real but

explainable:

1. The "10s-100s of TB" was about **build-time working space**, not final size — inserting billions of entries into redb in non-sorted order causes enormous write amplification and fragmentation. That's the I/O cost, separate from the resting footprint.
2. casey's number described the **original pre-0.16 design**, which stored more than a lean outputpoint→ranges table (extra/reverse indexes, less efficient encoding).
3. He explicitly hedged it ("if I recall correctly").

So the honest bottom line: the **marginal sat-range data for spent outputs is probably a few hundred GB to ~1 TB** on a modern encoding — but *building* it is the brutal part (write amplification and RAM), and that build cost, not the resting size, is the main reason the feature was pulled rather than made default.

One caveat worth stating: the "~2 ranges per output" average is the softest number in this estimate, and it's the one the total is most sensitive to. If you want, I can write a quick script to compute the actual distribution of ranges-per-output over a sample of historical blocks from your node — that would turn this from an order-of-magnitude estimate into a real figure. Want that?

explain the build cost. is it unavoidable?

Dissected database architecture tradeoffs between fundamental constraints and design artifacts >

The build cost comes almost entirely from the *write pattern*, not the data volume itself. Here's the mechanism.

What the build is doing

Indexing sats is a single sequential pass over the chain, forward-carrying ranges: to assign sat ranges to the outputs of block N, ord looks up the ranges of the inputs being spent (which are outputs from earlier blocks), slices them FIFO, and writes the resulting ranges to the new outputs. The lookup table is

`OUTPOINT_TO_SAT_RANGES`, stored in redb — which is a copy-on-write B-tree, keyed by outpoint.

The critical detail is that **outpoints are keyed by txid, and txids are hashes — effectively random**. So every output you write lands at a random position in a huge B-tree.

Why retention specifically explodes

In normal (pruning) mode, when an output is spent ord *deletes* its entry after carrying the ranges forward. So the live table never grows past roughly the UTXO set — ~180M entries, bounded, and a big fraction stays in RAM/page cache. Random inserts are cheap when the working set is cached.

Turn on retention and you stop deleting. The table now grows monotonically toward billions of entries — 10-15× larger — and blows past any reasonable page cache. At that point each random insert means: read a leaf page from disk (random read), modify it, write a new page back (copy-on-write), and rewrite parent pointers up the tree. A 58-byte logical insert turns into kilobytes of *random* disk I/O. That's write amplification, and it's happening billions of times against a tree too big to cache. The 256GB RAM recommendation exists purely to keep enough of that tree resident to stop the build from becoming IOPS-bound — even fast NVMe tops out around 10^5 - 10^6 random IOPS, and billions of amplified random ops at that rate is effectively forever. Add redb's COW behavior plus the savepoints ord keeps for reorg safety, and the *file* transiently bloats far past the resting size before it settles — which is where casey's "10s to 100s of terabytes to build" comes from, as opposed to the few-hundred-GB resting footprint.

Here's the insight that makes the cost diagnosable: **retention doesn't change the read side at all**. Forward-carry only ever looks up *currently-unspent* outputs (you can only spend something that's still unspent), so the lookup working set stays UTXO-sized whether or not you retain. Retention only inflates what you *write and keep*. The whole blowup is on the write/retain path, sitting in one hot random-access

keep. The whole story up to the write/retain path, ending in one not random access B-tree.

Is it unavoidable?

The fundamentals are unavoidable: you must do one linear pass over ~2.5B outputs (compute), and you must write the resulting few hundred GB to ~1TB once (storage). No way around those.

But the *pathological* part — the random I/O, the write amplification, the 256GB RAM — is an artifact of how ord retains, not a law of nature. It comes from leaving every spent entry inside the live, online, random-keyed B-tree. Cheaper designs exist:

- **Stream spent entries to cold sorted storage instead of leaving them hot.**
When normal mode prunes a spent output, instead of deleting it, append it to a batched, sorted-by-key archive. Sequential appends, not random B-tree churn. Query it later by binary search or bulk-load it into an outpoint-keyed structure at the end. This makes retention nearly free at build time.
- **Sort-then-bulk-load.** Collect the records, external-sort by outpoint, and build the B-tree bottom-up with sequential writes — the standard way to construct a large index, which sidesteps random insertion entirely.
- **Use an LSM engine** (RocksDB-style) instead of a B-tree. LSM trees turn random inserts into sequential SSTable flushes plus background compaction — they're built for exactly this write-heavy bulk-load profile. ord chose redb for single-file simplicity, good read performance, and clean MVCC for reorg handling, which is reasonable for a tip-following index but is the wrong shape for a one-time historical write storm.

So the honest answer is: the data and compute are unavoidable, but the brutal build cost is largely self-inflicted by the storage approach. The reason ord paid the full price is that `--index-spent-sats` was implemented as a one-line "stop pruning" toggle against the existing online B-tree, not as a re-architected bulk loader — which is exactly why it shipped as experimental and was later dropped rather than optimized. If you actually need this dataset, building it yourself with sorted bulk-loading or an LSM store is the route that avoids the pain, and it's why I keep nudging toward computing the ranges directly rather than resurrecting that flag.

why do you need more build space for saving STXO sat ranges, but you don't need the build space for saving just UTXO sat ranges

Analyzed live index size impact on build overhead costs >

Because the build overhead scales with the size of the *live, actively-mutated* index at each moment of the sync — not with how many records you ultimately keep. Pruning caps that live index at the UTXO set; retention lets it grow to billions. That single difference is the whole story, and it's worth seeing why.

Start with the counterintuitive bit: normal mode does *more* total operations than retention mode. It inserts every output when created **and** deletes it when spent — roughly $2N$ operations. Retention inserts every output and never deletes — roughly N . So retention does *fewer* writes and is still vastly more expensive. That tells you the cost isn't in the operation *count*. It's the cost *per* operation.

Cost per B-tree operation is cheap when the tree fits in cache and brutal when it doesn't. A logical insert against a resident tree just dirties an already-loaded leaf page. The same insert against an out-of-cache tree means a random disk read to fetch the leaf, a copy-on-write rewrite of that page, and rewrites of parent pointers up a deeper tree — kilobytes of random I/O for a 58-byte record.

Now apply that to the two modes:

Pruning mode — at every instant the table equals the *current* UTXO set. An output is inserted when created and deleted when spent, so it's never co-resident with the billions of others; the tree at block height t is just the live UTXO set at t . That was a few million entries for most of Bitcoin's history and ~180M now — small enough to stay cache-resident throughout. Inserts and deletes hit resident pages, freed pages get recycled immediately, savepoints pin only a handful of pages, fragmentation stays low. The file sits at roughly its resting size the entire build. There's no build-space blowup because there's nothing to blow up.

Retention mode — the table equals *every output created so far*, growing monotonically toward billions. Partway through the sync it exceeds RAM and never comes back. From there, the build-space overhead comes from four things, all multiplied by the now-enormous per-commit churn:

- Random-keyed (txid) inserts each trigger a random read + COW page rewrite + parent rewrites against a deeper tree.
- COW frees pages faster than the allocator reuses them, so the file grows past the live data.
- Savepoints kept for reorg safety pin old versions of a heavily-churned huge tree, inflating the file between compactions.
- Random inserts leave leaf pages half-full and scattered — fragmentation, so logical file size exceeds packed data.

So the asymmetry isn't "more bytes." The *resting* data difference really is just more bytes (~20GB of UTXO sat ranges vs a few hundred GB for STXO). The *build-space* difference is a different quantity entirely: it's overhead that scales with the size of the tree you're mutating, and pruning is precisely the mechanism that holds that size down. The build-space cost is small in UTXO mode not because you write less data overall — you write more — but because you never let the working structure grow past what fits in cache. Take pruning away and the overhead that was negligible becomes the dominant term.

Which loops back to the earlier point: this is a property of mutating one online B-tree, not of the data. If you streamed the spent entries out to sorted cold storage as you pruned them, the live tree would stay UTXO-bounded, the churn would stay small, and the build-space blowup would vanish — you'd pay only the one-time sequential cost of writing the archive.

with the regular sats indexer, doesn't the sat range of each STXO get calculated anyway and then pruned? why not just save it?

Excavated the cost distinction between computation and persistent storage placement >

You've put your finger on exactly the right thing — and the answer is that you're correct about the part you think you're correct about. The range is computed for free during forward-carry, and it's even better than that: at the moment an output is spent, its sat range is already physically sitting in the table. It was written there when the output was created, and it stays there as long as it's a UTXO. So "retain" doesn't mean "do extra work to save it." It literally means "skip the delete." At that instant, retaining is *cheaper* than pruning, because pruning is an operation and retaining is the absence of one.

So the cost of saving it is not paid when you save it. It's paid by everything that happens afterward.

Pruning's purpose was never to avoid writing the range — the range is already written. Pruning exists to keep the *tree small*. The deletes are a small, continuous premium ord pays to cap the working set at UTXO-set size, so that every subsequent insert, lookup, and commit operates on a structure that stays cache-resident. Skip the deletes and you pocket that small premium — but you forfeit the bound. The tree now grows without limit, and an unbounded, random-keyed, out-of-cache B-tree is what makes every *later* operation pay the write-amplification tax. The expense of retention is entirely downstream, levied on the operations that come after, not on the entry you kept.

And here's the part that really vindicates your instinct: those retained spent entries are never read again for the rest of the build. Forward-carry only ever looks up *currently-unspent* inputs, so once an output is spent and retained, the sync never touches it again. It just sits in the hot tree as dead weight — evicting live UTXO pages from cache, deepening the tree, fragmenting the file — without ever being useful until query time. It's not just kept; it's kept in the worst possible place.

Which is the whole resolution. Your intuition — "the data's right there, just save it" — is right. The error isn't saving it; it's saving it *into the live index*. If, at the moment you'd prune, you instead append the range to a separate sequential cold store (and still delete it from the live tree as usual), then: the hot tree stays UTXO-bounded, the append is sequential so there's no write amplification, and you've captured the data essentially for free. Same bytes, different location, completely different cost.

So "why not just save it" has a clean answer: you can, and nearly for free — but only if you save it somewhere other than the structure that has to stay small to stay fast.

ord's `--index-spent-sats` saved it in the one place that's ruinous, because it was

implemented as "stop deleting" rather than "divert to cold storage." The cost was about *where*, not *whether*.

where did index-spent-sats store it? why was it ruinous? so we can solve the problem just be storing it somewhere else?

Fetching source code to verify table names and storage details >

Let me verify where it actually stored them rather than keep inferring — this is a concrete implementation detail worth getting right.

Verified storage mechanism and planned source code inspection >

Let me pull the actual diff so I can tell you precisely where it put the data.

Verified storage mechanism and architected alternative solution >

Got the actual code — and it confirms the picture exactly. Here are the receipts.

Where it stored it: the same table as the live UTXO ranges.

ord defines a single table for sat ranges:

```
define_table! { OUTPOINT_TO_SAT_RANGES, &OutPointValue, &[u8] }
```

Key is the 36-byte outpoint, value is the sat ranges packed as consecutive 11-byte chunks (`chunks_exact(11)` in the code — which also confirms the 11-bytes-per-range figure from the size estimate). Both unspent and retained-spent outputs live in this one table. There is no separate store.

The flag is, almost literally, a one-token change in the spend path of the updater. When a transaction spends an input, ord does this:

```
if index.index_spent_sats {
    outpoint_to_sat_ranges.get(&key)?      // read, leave it in place
} else {
    outpoint_to_sat_ranges.remove(&key)?   // read AND delete (prune)
}
```

That's the entire mechanism. Normal mode calls `remove()` — it reads the input's ranges to carry them forward and deletes the entry in the same operation. With `--index-spent-sats`, it calls `get()` instead — reads the ranges, leaves the entry sitting in the table forever. Same table, same random (txid) key, same hot online B-tree the indexer mutates every block.

Why that's ruinous is now precise rather than inferred: the retained entries pile up in the exact structure that needs to stay small to stay cache-resident. Worse, look at what they are — a spent output can never be spent again, so once retained, those entries are *never read again during indexing*. They're pure dead weight: billions of never-touched records crowding the live UTXO working set out of cache, deepening the tree, and bloating the COW file. The flag took the data ord was already throwing away and, instead of throwing it away, stuffed it into the worst possible location.

So yes — relocating it solves the problem, and the fix is almost embarrassingly direct.

Notice that in normal mode, `remove()` *returns the value it deletes*. So at the moment of pruning, ord already has the spent output's sat ranges in hand, for free, as the return value of a call it already makes. The minimal fix is:

- Keep calling `remove()` exactly as normal mode does, so the live

`OUTPOINT_TO_SAT_RANGES` tree stays UTXO bounded and indexing stays at full

`OUTPUTPOINT_TO_SAT_RANGES` tree stays O(1) XO-bounded and indexing runs at run normal-`--index-sats` speed.

- Take the ranges that `remove()` hands back and append them to a *separate*, write-optimized sink — a sequential append-only log (sorted/bulk-loaded later for outpoint lookup), or an LSM store keyed by outpoint. Sequential writes, no random B-tree churn, trivial RAM.

The build cost then collapses to roughly "normal `--index-sats`" plus a cheap streaming sequential write of a few hundred GB. Queries for spent outputs hit the archive; queries for live outputs hit the existing table. The only real wrinkle is reorg safety — you don't want to permanently archive ranges for blocks that later get reorged out — but that's handled by buffering the last N blocks' worth of spent entries (in memory or in the existing savepoint window) and only sealing them into the immutable archive once they're buried deep enough to be reorg-safe.

So your instinct across the last few questions was right the whole way: the data is free to capture, the writing of it is cheap if it's sequential, and the entire 10TB/256GB horror story came from *where* `--index-spent-sats` put it, not from the existence of the data. ord paid the full price because it implemented retention as "don't delete" against the hot tree rather than "delete as usual, but divert the corpse to cold storage." Relocate the write and the problem essentially goes away.

If you want, I can sketch what that diversion would look like as a concrete patch against the updater's spend loop — where to tap the `remove()` return value, a simple sorted-segment archive format, and the reorg buffer — so you'd have a real starting point rather than just the shape of it.



